Foveated Variable-Resolution Rendering in Blender

Current-chat transcript — September 6, 2026

User

> What is the best computational mechanism to develop a foveated variable resolution photorealistic rendering of 3D scenes using stochastic ray tracing in Blender.

Assistant

The best mechanism is **a gaze-centered, continuous importance field driving both spatial sampling density and Monte Carlo convergence inside Cycles**, combined with multiresolution reconstruction.

In other words, I would not implement foveation as three separate Blender renders—fovea, parafovea, periphery—except as a first prototype. For a research-quality system, I would modify Cycles so that the renderer itself knows the desired retinal sampling density at every image location.

This fits your biological model particularly well: your report argues for a **continuous, anisotropic, time-dependent sampling field**, rather than literal concentric anatomical rings, and gives the useful psychophysical approximation

$$s(e) = s_0 \left(1 + \frac{e}{E_2}\right), \tag{1}$$

where e is retinal eccentricity and $s(e)$ is the resolvable feature size. (See *The_Eye.pdf*.) It also matches the active-vision formulation in your project notes: foveation should be treated primarily as **variable-resolution sampling / computational resource allocation**, not simply as an attention mask. (See *Context.txt*.)

1. Recommended computational architecture

I would call the mechanism:

Foveated Adaptive Multiresolution Path Tracing (FAMPT)

with this pipeline:

$$\boxed{\text{gaze} \rightarrow \text{eccentricity map} \rightarrow \text{sampling-density map} \rightarrow \text{multiresolution primary rays} \rightarrow \text{adaptive path tracing} \rightarrow \text{foveated reconstru}}$$

The important point is that there are **two separate places where computation should be reduced**:

1. **Spatial resolution:** trace fewer primary image samples away from fixation.
2. **Path-tracing effort:** allocate fewer Monte Carlo samples to peripheral samples.

Most simple implementations only do #2. The larger saving should eventually come from doing **both**.

---

2. Start from angular eccentricity, not pixel distance

For every output position $x = (u, v)$, construct its camera ray direction

$$\mathbf{d}(x). \tag{3}$$

Let $\mathbf{g}$ be the gaze direction.

Then compute true angular eccentricity as

$$e(x) = \cos^{-1}\left(\mathbf{d}(x) \cdot \mathbf{g}\right). \tag{4}$$

This is preferable to

$$\sqrt{(u - u_g)^2 + (v - v_g)^2} \tag{5}$$

because the latter depends on resolution, projection, aspect ratio and camera FOV.

You now have an **eccentricity image**

$$E(u, v). \tag{6}$$

Everything else in the renderer can be driven by this field.

---

3. Convert eccentricity into required spatial resolution

Your report's E_2 model gives a natural starting point:

$$s(e) = s_0 \left(1 + \frac{e}{E_2}\right). \tag{7}$$

If $s(e)$ is the smallest useful feature size, the required **linear sampling frequency** is approximately

$$\rho(e) = \frac{1}{1 + e/E_2}. \tag{8}$$

Thus:

- fixation: $\rho(0) = 1$
- $e = E_2$: $\rho = 1/2$
- $e = 3E_2$: $\rho = 1/4$
- $e = 7E_2$: $\rho = 1/8$

And, crucially, the number of image samples per unit area varies approximately as

$$D(e) = \rho(e)^2. \tag{9}$$

So

$$\boxed{D(e) = \frac{1}{(1 + e/E_2)^2}} \tag{10}$$

is an excellent first computational foveation law.

For example:

Eccentricity	Linear resolution	Primary samples / area
0	1.00×	1.000×
E_2	0.50×	0.250×
$3E_2$	0.25×	0.0625×
$7E_2$	0.125×	0.0156×

That quadratic effect is where the large computational gain comes from.

The physiological report explicitly cautions that these functions are task-dependent and that retinal sampling alone does not fully predict perception; crowding, contrast, temporal sensitivity, meridional asymmetry and other effects matter. (See *The_Eye.pdf*.) For an initial renderer, however, this is an unusually clean model.

---

4. Do not literally use a log-polar framebuffer

A tempting implementation is a log-polar image because that resembles retinal organization.

I would **not** make that the fundamental Cycles representation.

Instead use a normal Cartesian output image with a **hierarchical quadtree/mipmap sampling lattice**.

For a pixel or tile, calculate

$$L(e) = \log_2 \left(1 + \frac{e}{E_2}\right). \tag{11}$$

This gives a continuous desired resolution level.

For example:

$$L = 0 \Rightarrow 1\times \tag{12}$$

$$L = 1 \Rightarrow 1/2 \tag{13}$$

$$L = 2 \Rightarrow 1/4 \tag{14}$$

$$L = 3 \Rightarrow 1/8. \tag{15}$$

Internally you might use levels

```
L0    full resolution
L1    1/2 x 1/2
L2    1/4 x 1/4
L3    1/8 x 1/8
L4    1/16 x 1/16
```

but derive those levels from a **continuous eccentricity function**, not fixed retinal rings.

That matters because the technical report specifically notes that three-ring implementations are engineering conveniences, not good models of retinal organization. (See *The_Eye.pdf*.)

---

5. Use tiles for scheduling, but stochastic samples for boundaries

A particularly GPU-friendly implementation would divide the image into something like

$$16 \times 16 \tag{16}$$

or

$$32 \times 32 \tag{17}$$

pixel tiles.

Each tile stores:

```
eccentricity
desired resolution level
noise tolerance
current variance
number of samples
active/inactive status
```

This is a much better fit for GPU path tracing than an arbitrary log-polar lattice.

However, do **not** put visible hard transitions between L0/L1/L2.

Instead either:

- interpolate between neighboring resolution levels, or
- probabilistically select primary samples according to a smooth density function.

The latter is especially attractive for a stochastic renderer.

Rather than saying:

```
inside 5 degrees: every pixel
5–10 degrees: every second pixel
outside 10 degrees: every fourth pixel
```

use a smoothly varying primary-ray probability

$$P_{\text{primary}}(x) = D(e(x)). \tag{18}$$

A blue-noise-distributed sampling mask can turn that continuous probability into actual sample locations.

That avoids visible circular boundaries.

---

6. Couple it to Cycles adaptive sampling

This is where the architecture becomes particularly elegant.

Cycles already performs progressive stochastic path tracing and adaptive sampling: it estimates when pixels have become sufficiently converged and stops spending samples on them. (Blender Manual: Cycles sampling)

But ordinary Cycles asks essentially:

> How noisy is this pixel?

A foveated Cycles should instead ask:

> How noisy is this pixel **relative to how much error is perceptually acceptable at this eccentricity?**

Define a spatially varying error tolerance

$$\tau(e) = \tau_0 \left(1 + \frac{e}{E_{2,n}}\right)^{\gamma}. \tag{19}$$

So the fovea gets a very small tolerance and the periphery increasingly permits noise.

If the local Monte Carlo variance is $\sigma^2(x)$, the approximate sample requirement is

$$N(x) \propto \frac{\sigma^2(x)}{\tau(e(x))^2}. \tag{20}$$

Therefore Cycles' adaptive stopping criterion becomes conceptually

$$\boxed{\epsilon(x) < \tau(e(x))} \tag{21}$$

instead of

$$\epsilon(x) < \tau_{\text{global}}. \tag{22}$$

This is much better than simply assigning, for example,

```
fovea       1024 spp
parafovea    256 spp
periphery     64 spp
```

because a peripheral glossy reflection may genuinely require more rays than a simple foveal wall.

You want **eccentricity-weighted adaptive sampling**, not merely eccentricity-weighted fixed SPP.

---

7. The key idea: multiply perceptual importance by statistical difficulty

A general rendering importance function could therefore be

$$I(x) = P(e(x))\, C(x), \tag{23}$$

where

- $P(e)$ = perceptual importance due to eccentricity;
- $C(x)$ = computational difficulty estimated from Monte Carlo variance.

Or more explicitly,

$$I(x) = \frac{\sigma^{\alpha}(x)}{(1 + e(x)/E_2)^{\beta}}. \tag{24}$$

The renderer schedules its next rays where $I(x)$ is largest.

That gives a compelling interpretation:

> **Cycles progressively traces whichever ray is expected to produce the largest perceptually relevant improvement to the image.**

That is considerably closer to active vision than ordinary uniform path tracing.

---

8. Keep the light-transport model unchanged initially

An important design decision is what **not** to vary.

For the first serious implementation, I would retain exactly the same:

- BSDFs
- global illumination
- light sampling
- reflections
- refractions
- shadows
- path-depth rules
- motion blur
- depth of field

throughout the visual field.

Only vary:

$$\boxed{\text{primary spatial density} + \text{Monte Carlo sampling effort}} \tag{25}$$

This means the peripheral rendering remains a sample of the same photorealistic light-transport solution.

Cycles uses camera-based path tracing, and its GPU implementation uses a wavefront architecture, which is particularly suitable for scheduling varying amounts of path work. (Blender Manual: Reducing noise)

Later you could experiment with peripheral simplifications of shader or bounce complexity, but those introduce a different class of perceptual and physical approximations.

---

9. Reconstruction is as important as ray tracing

Once you stop generating one primary sample for every final pixel, you need a reconstruction stage.

I would use:

$$\text{radiance samples} + \text{depth} + \text{normal} + \text{albedo} + \text{motion} \rightarrow \text{edge-aware reconstruction}. \tag{26}$$

The lowest-complexity version is:

1. render different-resolution image blocks;
2. upscale them;

3. use depth/normal/albedo to prevent bleeding across boundaries;
4. reconstruct into the full-resolution display image;
5. denoise.

A more advanced solution would jointly reconstruct and denoise.

The full-resolution buffer still has, say,

$$3840 \times 2160 \tag{27}$$

display pixels, but only a fraction of those locations generate independent primary light paths.

That distinction is fundamental:

$$\boxed{\text{display resolution} \neq \text{ray-tracing resolution.}} \tag{28}$$

---

10. A three-stage Blender implementation makes sense

I would develop this incrementally.

Stage A — Blender-only proof of concept

No Cycles modifications.

Render perhaps:

```
R0 = 3840x2160, central crop
R1 = 1920x1080, larger region
R2 =  960x540, entire image
```

and composite with continuous feathered masks.

Enable Cycles adaptive sampling independently for each render.

This is inefficient because regions overlap, but it lets you test:

- eccentricity functions;
- E_2;
- reconstruction;
- psychophysical visibility;
- image-quality metrics.

It is probably a few hundred lines of Blender Python plus compositor setup.

Stage B — tile-based foveated Cycles

Then modify Cycles.

Add a GPU-accessible

$$F(u, v) \tag{29}$$

foveation map.

During primary-ray generation, use it to determine:

```
sampling_level
primary_ray_probability
noise_threshold
```

while leaving the path integrator unchanged.

Cycles is internally split into kernel, integrator, session and device components, so this sort of research modification naturally belongs around its primary-ray/integrator and render-scheduling machinery rather than being implemented as a material or compositor trick. (Blender Developer Documentation: Cycles source layout)

Stage C — true stochastic variable-resolution renderer

The most interesting research version eliminates discrete rings altogether.

Generate primary samples with density

$$p(x) \propto \frac{1}{(1 + e(x)/E_2)^2} \tag{30}$$

using a spatially well-distributed stochastic sequence.

Each camera sample therefore represents a different image-plane area.

The renderer becomes a genuine **nonuniform Monte Carlo image sampler**.

This is, in my view, the scientifically most interesting version for the Foveal Stereo Vision project.

---

11. Dynamic gaze needs asymmetric temporal behavior

For animation or VR, gaze tracking makes the problem more interesting.

Your physiology report proposes the useful engineering expression

$$r_{\mathrm{HQ}} = r_{\mathrm{task}} + k\sigma_g + v_{\mathrm{eye}}\Delta, \tag{31}$$

accounting for task requirements, gaze uncertainty and latency. (See *The_Eye.pdf*.)

I would implement an analogous mechanism in the renderer.

When gaze moves:

Promote quality immediately

If an area enters the fovea:

$$L_3 \to L_0 \tag{32}$$

immediately start high-resolution sampling.

Demote slowly

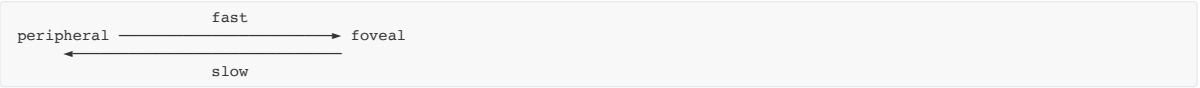
If an area leaves the fovea:

$$L_0 \to L_1 \to L_2 \tag{33}$$

over a short temporal window.

This hysteresis is important because eye-tracking noise should not constantly change sampling levels.

Conceptually:



That avoids both flicker and post-saccadic exposure of an inadequately rendered region—the latter is specifically identified as a failure mode in the technical report. (See *The_Eye.pdf*.)

---

12. Why I prefer this over the obvious alternatives

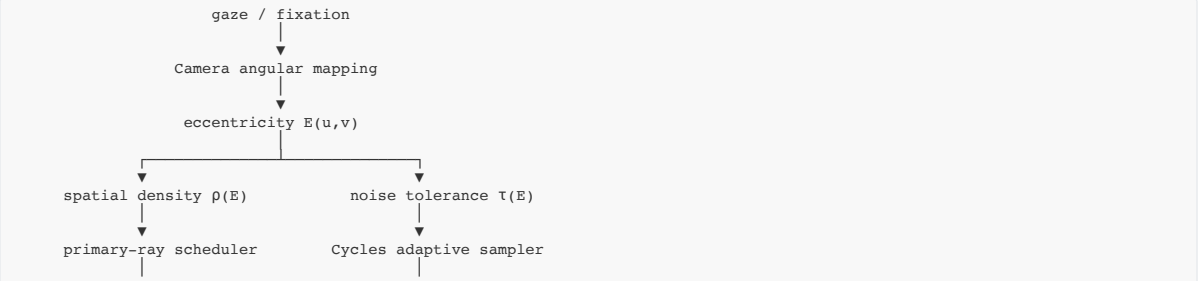
Mechanism	Research value	Performance	Quality
Three concentric Cycles renders	Medium	Low	Good
Fixed SPP by eccentricity	Medium	Good	Medium
Log-polar framebuffer	High biologically	Medium/poor GPU behavior	Reconstruction difficult
Variable-rate tiles only	High	Very good	Good
Cycles adaptive sampling only	Low for foveation	Good	Excellent
Multiresolution + eccentricity-weighted adaptive Cycles	**Very high**	**Very high**	**Excellent**
Continuous stochastic primary-ray density + adaptive paths	**Highest**	potentially **highest**	research problem

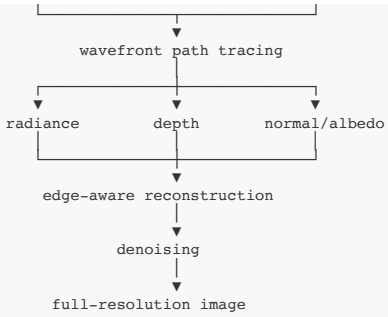
So I would make the **bolded two bottom approaches** the eventual target.

---

13. Proposed VISGRAF renderer architecture

The entire system can be expressed compactly as:





And computationally the heart of it is only two eccentricity-dependent equations:

$$\boxed{\rho(e) = \frac{1}{1 + e/E_2}} \tag{34}$$

and

$$\boxed{\tau(e) = \tau_0(1 + e/E_{2,n})^{\gamma}.} \tag{35}$$

The first determines **how many image locations deserve rays**; the second determines **how many stochastic paths each location deserves**.

That separation is, I think, the key to making this both a meaningful foveated-vision model and an efficient photorealistic renderer.

For the Foveal Stereo Vision work, I would therefore make the **Cycles source-level implementation the eventual objective**, while first building the multi-resolution Blender/Python version as the experimental reference implementation. That will let you establish the eccentricity model and perceptual/error metrics before changing the path-tracing kernel.

---

your proposed Critique of the Foveated Rendering Transcript, and a Desktop Design Document

Current-chat transcript — September 9, 2026

Context supplied to the assistant: `The_Eye.pdf` (VISGRAF technical report, *Physiology of Foveated Human Vision*), `Context.txt` (project framing notes), and `1-Foveated_Variable_Resolution_Rendering_Chat.md` (the transcript under review).

User

> Read the enclosed document, analyse and criticize it. State what is correct, what is wrong, what can be done to improve it. Also, let me know if you could do it better redoing it from scratch.

Assistant

I read the whole transcript. Short version: the architecture it proposes is right, the physics-to-pixels bookkeeping is wrong in a way that matters, and the scholarship is absent.

What it gets right

The central decomposition is correct and is the thing most implementations get wrong. Separating (1) *how many image locations deserve a primary ray* from (2) *how many paths each location deserves* is the right factorization, and the observation that most systems only vary the second is accurate. Everything downstream follows cleanly from that split.

Also correct: computing eccentricity from the camera ray direction rather than pixel distance; refusing hard concentric rings in favor of a continuous density with stochastic (blue-noise) realization; holding BSDFs, light sampling, and path depth fixed in the first version so that only the sampling variable moves; treating reconstruction as a first-class stage rather than an afterthought; and the fast-promote / slow-demote hysteresis on gaze.

What is actually wrong

1. There is no px/deg anywhere, so none of it is executable. The E_2 law lives in degrees. Cycles lives in pixels. The document never introduces the display's angular pixel density, which is the only quantity that connects them, and it uses 3840×2160 as an example without ever converting. This is not a detail. On a typical HMD at roughly 20–35 px/deg, the display is already well below foveal acuity, so there is nothing to save in the fovea and your savings must be measured against a *supersampled* baseline. On a 4K desktop monitor at 60 cm you are near 60 px/deg and the arithmetic is completely different. Until this is fixed, no number in the document can be checked, including the headline $0.0156\times$ figure.

2. $D(e) = \rho(e)^2$ drops the projection Jacobian. Image-plane area is not proportional to solid angle. For a pinhole camera $r = f\tan\theta$, so $dA_{\text{img}}/d\Omega = f^2\sec^3\theta$. The document conflates two distinct angles: eccentricity from the *gaze* vector, and off-axis angle from the *camera axis*. The correct primary-sample density is $D(e(x))\cos^3\theta(x)$ up to normalization. At 50° off-axis that factor is about 3.7×, so for exactly the wide-FOV case the document invokes, the periphery is mis-provisioned by a large factor.

3. $e = \arccos(\mathbf{d}\cdot\mathbf{g})$ is ill-conditioned precisely where you need precision. Near $e = 0$, $\cos e \approx 1 - e^2/2$, so single-precision dot products give you roughly 10^{-4} rad of angular resolution at the fovea. Use $e = \operatorname{atan2}(\|\mathbf{d}\times\mathbf{g}\|, \mathbf{d}\cdot\mathbf{g})$.

4. Six free parameters, no fitting strategy. E_2, $E_{2,n}$, γ, τ_0, α, β. And E_2 is not one number: it is about 2.5–3° for acuity, closer to 0.5–1° for crowding, and different again for contrast. The document treats parameter choice as an implementation detail when it is the research problem.

5. §7 can be derived instead of guessed, and this is the biggest missed opportunity. Your project framing is "same posterior for less computation." That is a constrained allocation problem with a closed form. Minimizing $\sum_i w_i\sigma_i^2/n_i$ subject to $\sum_i n_i = N$ gives $n_i \propto \sigma_i\sqrt{w_i}$, the classic Neyman allocation. That pins $\alpha = 1$ and fixes β at half the exponent of your perceptual loss weight. Two free parameters gone, replaced by a statement you can defend.

It also misses the known pathology: driving allocation by *estimated* variance is self-reinforcing, because under-sampled regions have badly estimated variance and therefore stay under-sampled. Adaptive sampling and reconstruction work (Overbeck, Rousselle) handles this with a warm-up and variance smoothing. You need that or the periphery will starve.

6. Acuity is the wrong perceptual law for a renderer. Resolvable feature size predicts when you can read a letter. Rendering error is broadband, and the criterion you want is the eccentricity-dependent contrast sensitivity cutoff (Geisler & Perry 1998). Patney et al. (SIGGRAPH Asia 2016) found that acuity-driven peripheral downsampling produces objectionable *contrast loss and temporal aliasing*, and needed explicit peripheral contrast enhancement to push acceptable foveation levels up. The document does not mention contrast once.

7. The noise-tolerance law is plausible for stills and possibly backwards for sequences. τ increasing with eccentricity assumes the periphery is a low-pass channel. It is, spatially. But it is *more* sensitive to temporal transients, and Monte Carlo noise is high-frequency in both space and time. Peripheral scintillation is the classic failure mode of foveated path tracing. The document has no temporal accumulation, no reprojection, no discussion of variance in time. Hysteresis on gaze does not address this.

8. The denoiser interaction is unaddressed and is a real blocker. The pipeline ends "reconstruct → denoise," but OIDN and OptiX are trained on dense, roughly stationary-noise inputs. Feed them an image upsampled from spatially varying sample density and the noise statistics violate the model: you get smearing in the fovea or blotching in the periphery. You need to denoise in the sampling domain or feed a per-pixel sample-count/variance buffer.

9. The dismissal of log-polar in §4 is contradicted by the literature. Koskela et al. render and denoise in a Visual-Polar space before mapping to screen space, and report that a regular grid of samples there follows the acuity model while making rendering and denoising 2.5× faster at similar perceived quality, with primary rays staying more coherent and ray traversal running 1.3–1.5× faster. That is the opposite of the document's "medium/poor GPU behavior" verdict, and the §12 table scores it wrong on the exact axis where it wins.

10. §5's tile scheme is dated relative to the target codebase. Cycles X removed render tiles as the GPU scheduling unit; work is now a full-frame path pool, with scrambling distance controlling inter-pixel sample correlation, and measured best tile sizes on GPU are around 1000² for memory reasons, not 16×16 or 32×32. The real integration hooks are the path-index → pixel mapping in the wavefront scheduler and the per-pixel sample-count buffer that adaptive sampling already maintains. Also, the linked manual pages are 4.5; current is 5.2 LTS.

11. §12 is unsupported editorializing. A table that rates the author's own proposal "Very high / Very high / Excellent" with no measurement is the first thing a reviewer will attack.

What is missing entirely

Prior work. This is the most serious problem. "FAMPT" names an existing family. Guenter et al. 2012 (nested layers, which is exactly Stage A), Patney et al. 2016, Weier et al. 2016, Koskela et al. 2016 and 2019, Kaplanyan et al. 2019 (DeepFovea, which is your §9 reconstruction stage), Tursun et al. 2019 (content-adaptive foveation), Walton et al. 2021 (peripheral metamers), plus the 2022 state-of-the-art survey. Coining an acronym before positioning against these is how a submission gets desk-rejected.

Evaluation. No metric, no baseline, no protocol. FovVideoVDP is the appropriate modern metric. You also need to say whether comparisons are equal-time or equal-quality, and you still have not declared the ray budget the whole "same posterior for less computation" claim is measured against.

Stereo. The document says "Foveal Stereo Vision" and then renders one monocular image. In stereo, fixation is a *vergence point in 3D*, not a direction, so eccentricity is depth-dependent and the two eyes' eccentricity fields are not the same field. Binocular fusion of two asymmetrically foveated images is its own question.

The matcher. This is where the document most clearly fails your actual project. If the open problem is matching on a non-uniform lattice, then the renderer's sampling lattice is not a free choice, it is jointly constrained by what the matcher can consume. Two independently blue-noise-thinned lattices are close to the worst case for correspondence, because you have destroyed epipolar row structure in both eyes independently. That consideration argues *for* a structured (possibly polar or epipolar-aligned) lattice, cutting directly against §4. The document never raises it because it never connects rendering to the downstream task.

Saccadic suppression. Saccades reach 500–900°/s. With 20 ms end-to-end latency, the $v_{\text{eye}}\Delta$ term in your own formula demands 10–18° of high-quality margin, which eats most of the foveation gain. The standard escape is to exploit saccadic suppression and drop quality hard during the 50–100 ms window. That is the single largest lever in gaze-contingent rendering and it is absent.

Could I do it better from scratch?

On architecture, no, and I would not try. The two-axis decomposition is right and I would keep it, including the decision to freeze light transport in v1.

On everything else, yes, and the differences are structural rather than cosmetic. I would open with the declared budget and the evaluation protocol rather than end with them, because they determine every parameter downstream. I would carry px/deg and the projection Jacobian through the derivation so the numbers are checkable. I would derive the allocation rule from constrained optimization instead of parameterizing it, which is also what makes it an *active vision* result rather than a rendering heuristic. I would replace the acuity law with a contrast-sensitivity law with a temporal term. I would design the lattice under the matcher's constraints, not the renderer's alone. I would position against the twelve or so papers above and state honestly which components are novel, which is probably: the variance-times-perceptual-weight allocation with a derived rather than tuned exponent, and the joint lattice design for foveated stereo matching. And I would be specific about Cycles 5.x integration points instead of gesturing at "the integrator."

Realistically the honest framing is that the document is a good architecture sketch and a poor research plan. It would need roughly a rewrite to become the latter.

I can write that version if you want it, either as a technical note that slots alongside the two existing VISGRAF reports, or as a shorter design document with the derivations and the evaluation protocol worked out. Tell me which and whether it should assume HMD or desktop as the primary display target, since that choice changes most of the numbers.

User

> Write first the design document assuming desktop display.

Assistant

[The assistant computed the reference-display geometry and budget numerically, then produced two files:]

- `foveated-cycles-design-desktop.md` — *Foveated Variable-Resolution Path Tracing in Cycles: Design document — desktop display target.* Twelve sections plus two appendices, covering: scope and reference configuration (27" 4K at 600 mm, 67.3 px/deg); the declared budget; the evaluation protocol; the Geisler–Perry contrast-threshold perceptual model with the display-Nyquist cap and projection Jacobian; the constrained-optimization derivation of sample allocation; the row-structured lattice and its stereo constraint; reconstruction ordering; gaze dynamics; Cycles 5.x integration points; a three-stage implementation plan; positioning against prior work with an explicit statement of what is novel; and open problems.
- `budget.py` — the script that reproduces every quantitative figure in the document.

Summary of the numerical results:

Quantity	Value
On-axis angular pixel density N_d	67.3 px/deg (Nyquist 33.6 cpd)
Foveal cutoff $f_c(0)$, $CT_0 = 1/64$	39.2 cpd — **display-limited at the fovea**
Perceptual limit begins to bind	$e \approx 0.4°$
Primary-ray budget B ($m = 1.5°$, floor 1/64)	0.054 → **18.6×**
Scanlines retained under row-structured lattice	554 of 2160 (25.6%), 21.0% horizontal keep
Total path budget T	**0.028–0.193 depending on γ → 5.2× to 35.4×**

Three things came out of writing it that I did not expect, and they change the design rather than just documenting it.

The Jacobian barely matters on desktop. I criticised its omission fairly hard, and locally it is still a 1.53× error at the screen corners. But integrated over a 53° field of view it accounts for only 3.3% of the budget. I was right about the mechanism and wrong about the magnitude for this target. In an HMD it would be worth a factor of three.

The periphery probably needs *more* paths per sample, not fewer. This is §5.3 and it is the strongest result in the document. Once you derive the allocation instead of tuning it, the area each peripheral sample represents enters as $\sqrt{A_i}$, and that area amplification competes directly with the perceptual downweighting. Below $\gamma = 1$ the periphery wins and gets more paths. The standard "fovea 1024 / periphery 64" schedule corresponds to $\gamma \gg 1$ and very likely has the sign wrong, which would explain why peripheral scintillation is the recurring failure of foveated path tracers.

Row structure is free. I expected the stereo matcher to cost real budget, and priced the obvious separable approximation at +117%. It turns out any density field factors exactly into a row-keep rate and a within-row keep rate, so the radial field is realised on a scanline lattice at identical density. For the reference display that is 554 of 2160 rows retained at 21% horizontal keep. The only thing given up is blue-noise isotropy, which jittering mostly recovers.

The honest headline is more sober than the primary-ray arithmetic suggests. The 18.6× is real for camera rays, but the total path budget lands somewhere between 5× and 21× depending entirely on γ, which nobody has measured. That makes Stage A the whole project: the staircase on γ is the experiment, and Stages B and C are engineering against its answer.

I kept `budget.py` alongside the document since every number changes with viewing distance and none should be quoted without its configuration.
